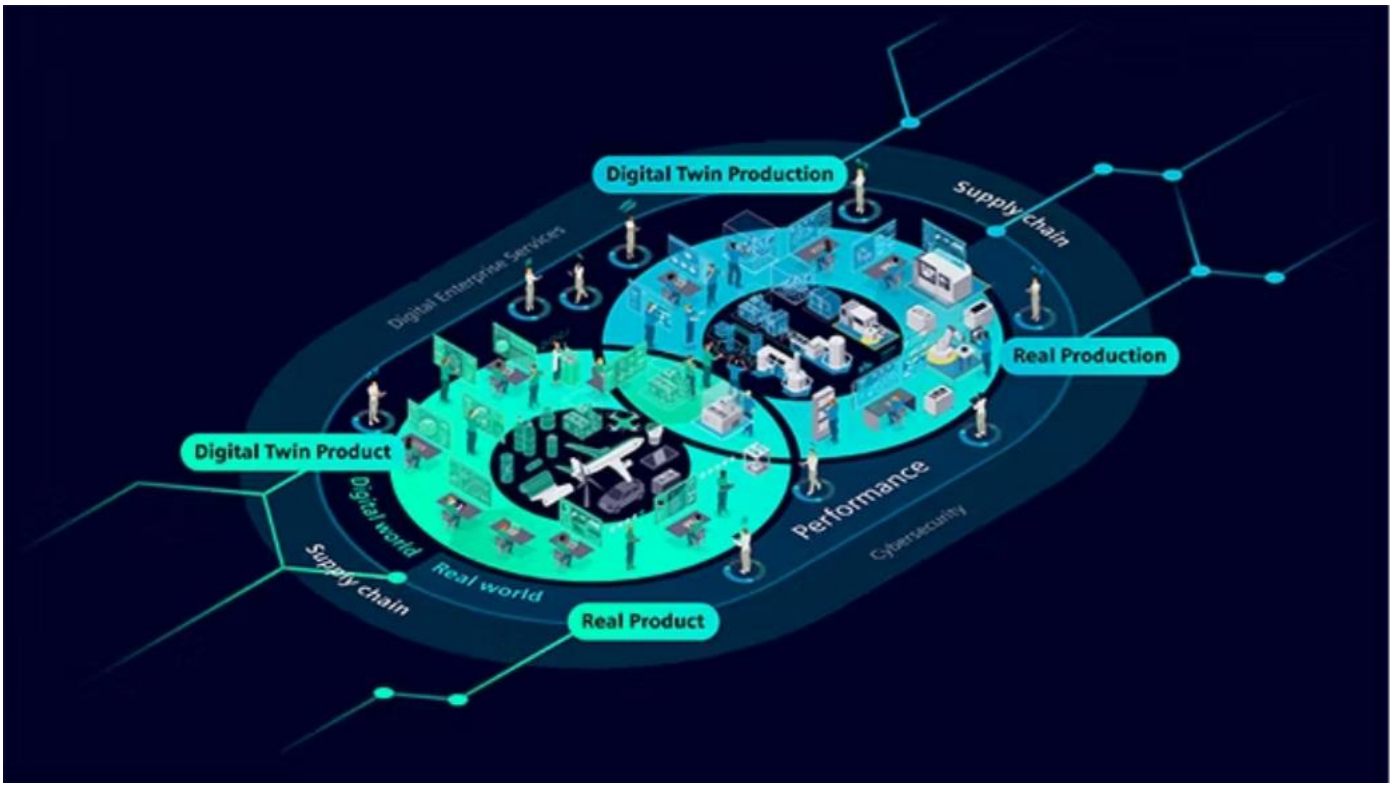
Line-Following Robot with PID Control



Submitted by:

Name: Mariam Amr Mohamed

Department: Electronics and Communication Engineering

Student ID: [2200704]

Course Details:

Course Name: [Digital Twin]

Instructor: [Mohamed AbdElSalam]

Overview:

A differential-drive robot spawns near a predefined line/path on a 2D plane. The controller steers the robot onto the path and tracks it to the end with minimal lateral error. The system consists of three lightweight clients connected over the VSI fabric:

- Client 1 - Simulator: Simulates robot kinematics and publishes pose
- Client 2 - Controller: PID-based control minimizing lateral error
- Client 3 - Plotter: Plots trajectory and logs KPIs

Robot Kinematics Model:

The differential-drive robot is modeled using the following kinematic equations, derived from the unicycle model:

$$\frac{dx}{dt} = v \cdot \cos(\theta)$$

$$\frac{dy}{dt} = v \cdot \sin(\theta)$$

$$\frac{d\theta}{dt} = \omega$$

Where x, y are the robot's position, θ is the heading angle, v is the linear velocity, and ω is the angular velocity (control output).

PID Controller:

$$\omega = Kp \cdot e + Ki \cdot \int edt + Kd \cdot \frac{de}{dt}$$

Where the lateral error is: $e = Ydesired - Ycurrent$

VSI Architecture:

The system uses three Python clients communicating over the Innexis Virtual System Interconnect CAN backplane.

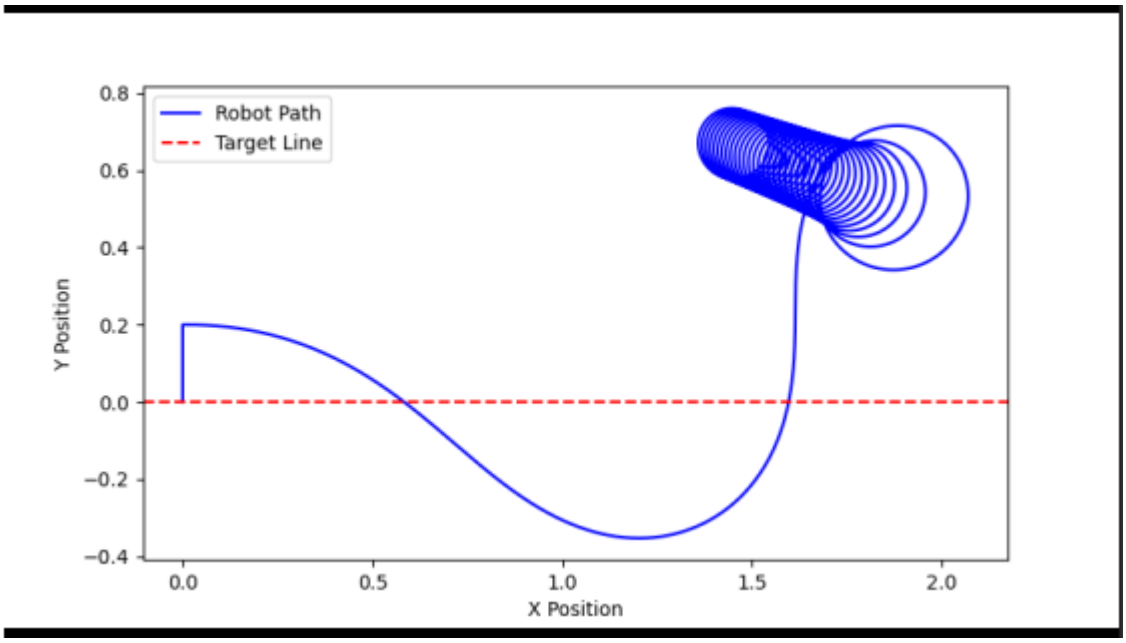
Client 0 (Simulator) runs at port 50101 and publishes pose_x, pose_y, pose_theta on CAN IDs 10, 11, 12.

Client 1 (Controller) runs at port 50102, subscribes to pose signals and publishes cmd_v, cmd_w on CAN IDs 20, 21.

Client 2 (Plotter) runs at port 50103 and subscribes to pose_x, pose_y for visualization and KPI logging.

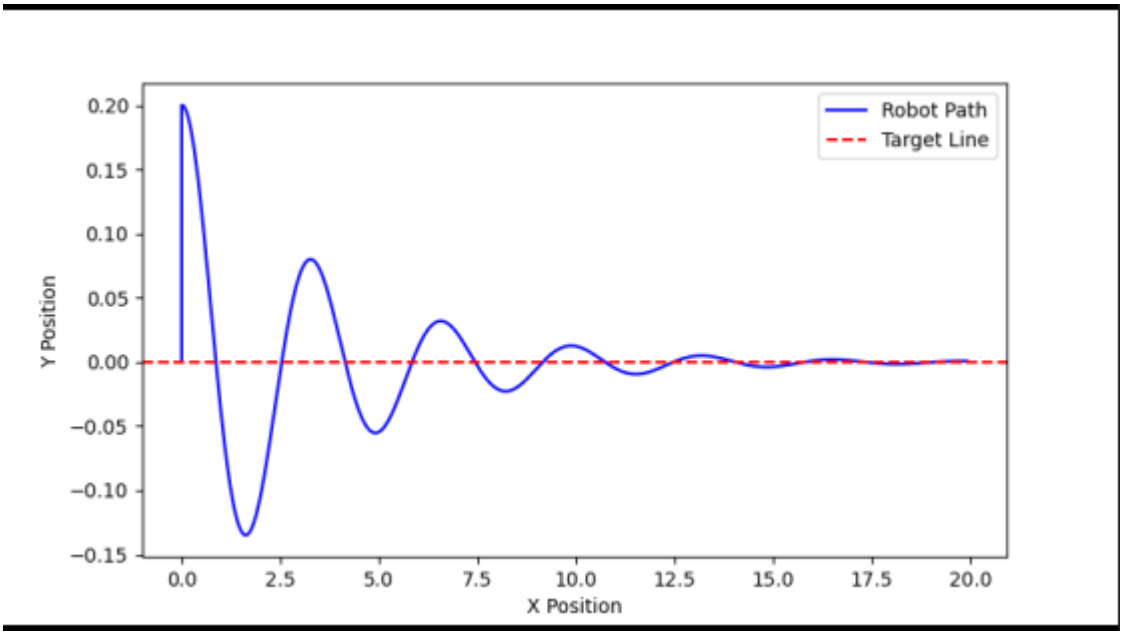
1. E1 - PID Gain Sweep:

- Kp =1.8 ki=0.02 kd=0.5



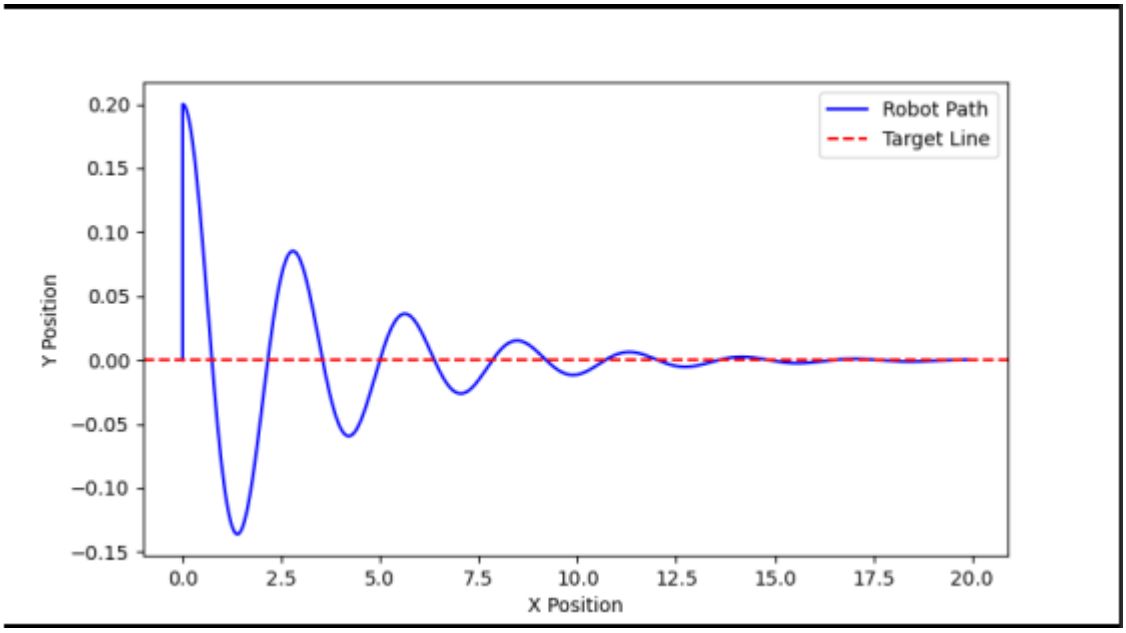

```
=== KPI Results ===
Overshoot: 0.3526 m
Steady State Error: 0.7077 m
Settling Time: Did not settle
```

- Kp =1.5 ki=0.001 kd=2




```
=== KPI Results ===
Overshoot: 0.1349 m
Steady State Error: 0.0007 m
Settling Time: 13.08 s
```

- Kp =2 ki=0.001 kd=4




```
=== KPI Results ===
Overshoot: 0.1361 m
Steady State Error: 0.0003 m
Settling Time: 11.42 s
```

Run	Kp	Ki	Kd	Overshoot	SS Error	Ts
1	1.8	0.02	0.5	0.3526 m	0.7077 m	Did not settle

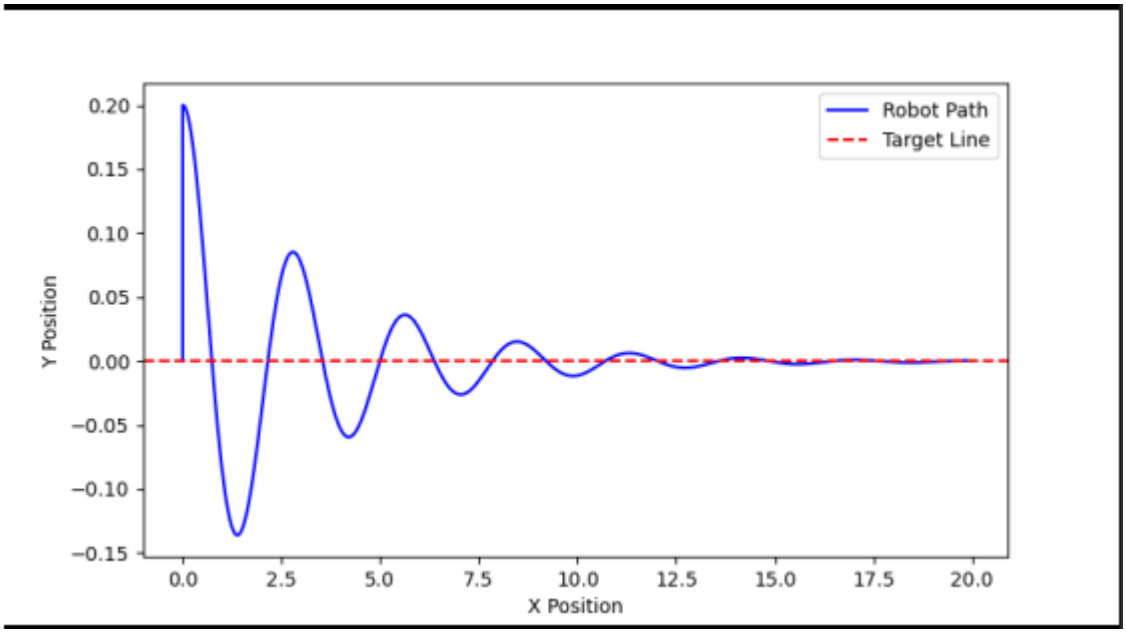
2	1.5	0.001	2.0	0.1349 m	0.0007 m	13.08 s
3	2.0	0.001	4.0	0.1361 m	0.0003 m	11.42 s

Discussion:

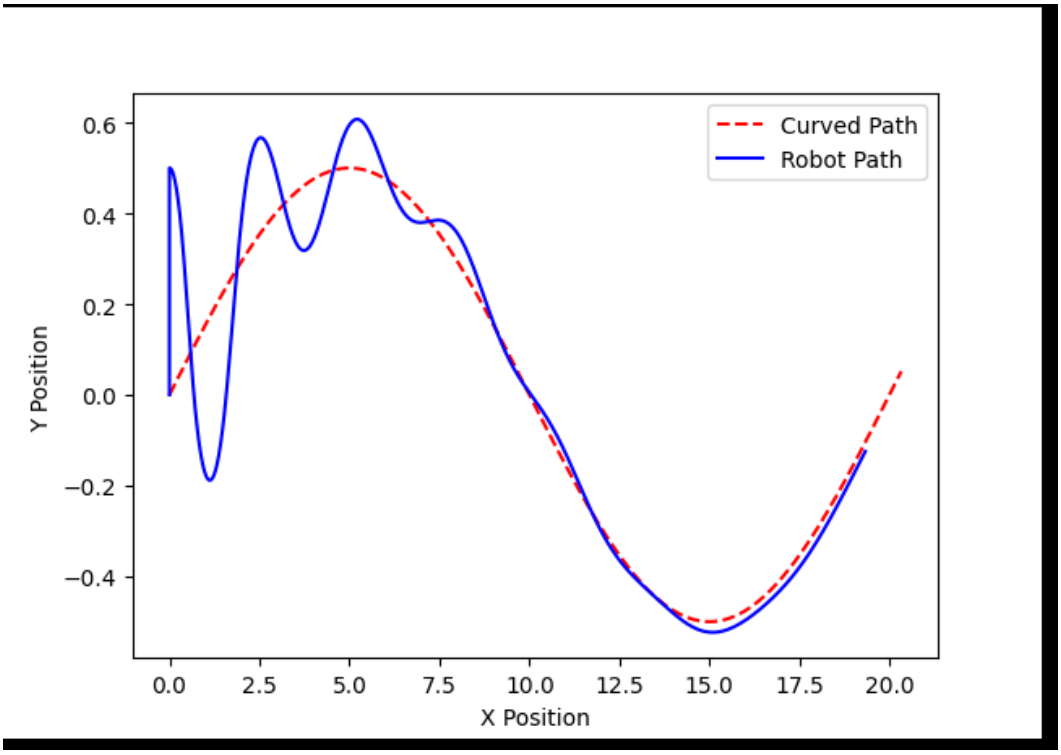
Run 1 (Kp=1.8, Ki=0.02, Kd=0.5) failed to settle because the low Kd value could not dampen oscillations, and the high Ki caused integral windup. Run 3 (Kp=2.0, Ki=0.001, Kd=4.0) achieved the best performance with the lowest settling time (11.42s) and steady-state error (0.0003m) due to the high Kd suppressing overshoot effectively.

2. E2 - Curved Path Robustness:

- Kp =2 ki=0.001 kd=4




```
=== KPI Results ===
Overshoot: 0.1361 m
Steady State Error: 0.0003 m
Settling Time: 11.42 s
```



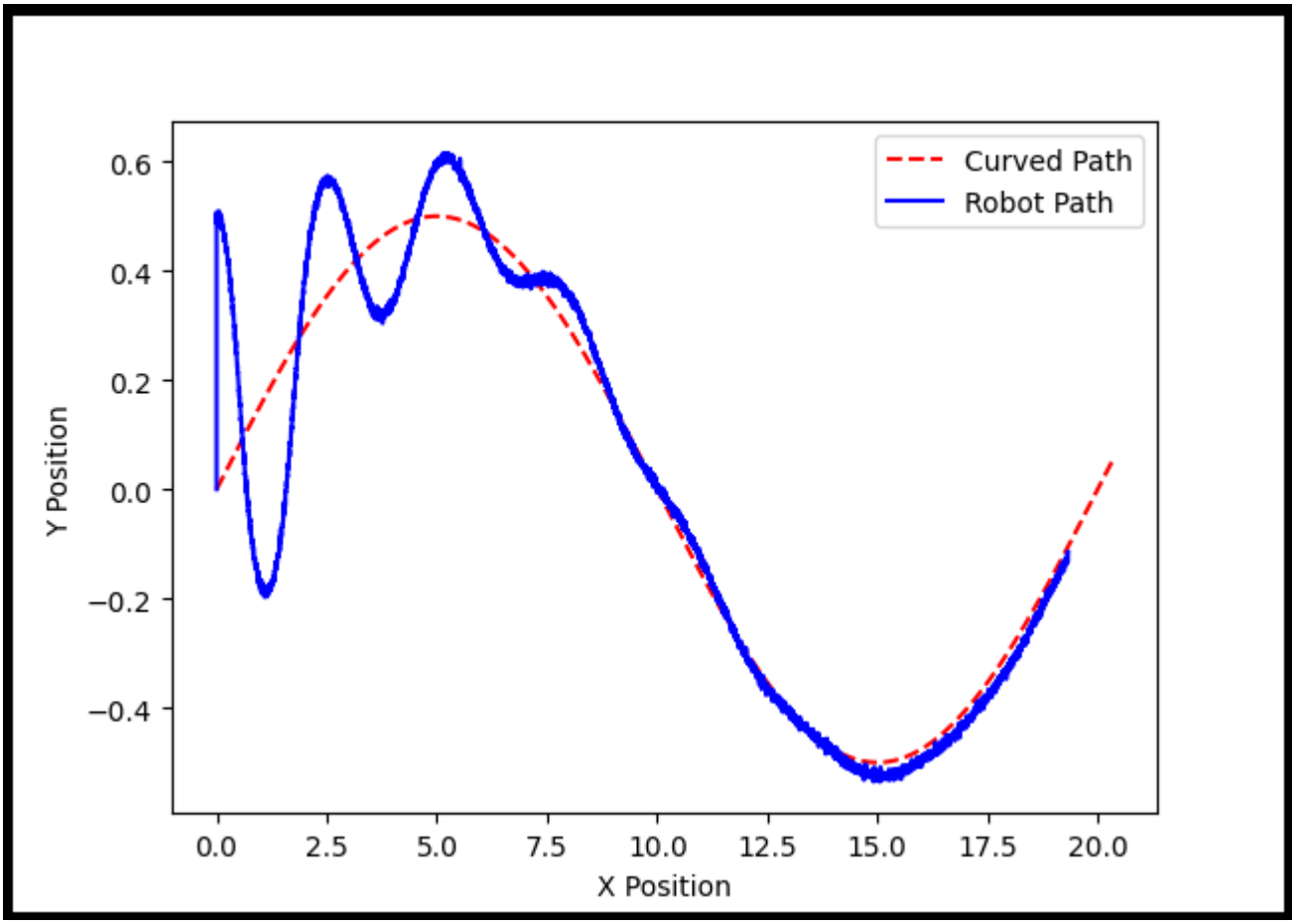

```
=== E2 KPI Results ===
Overshoot: 0.5000 m
Steady State Error: 0.0203 m
Settling Time: 22.45 s
```

Discussion:

On the curved path, the overshoot increased significantly from 0.1361m to 0.5000m compared to the straight path. This is because the desired_y is continuously changing, creating a persistent tracking error that the controller must constantly correct. The settling time also increased from 11.42s to 22.45s due to the dynamic nature of the curved reference signal.

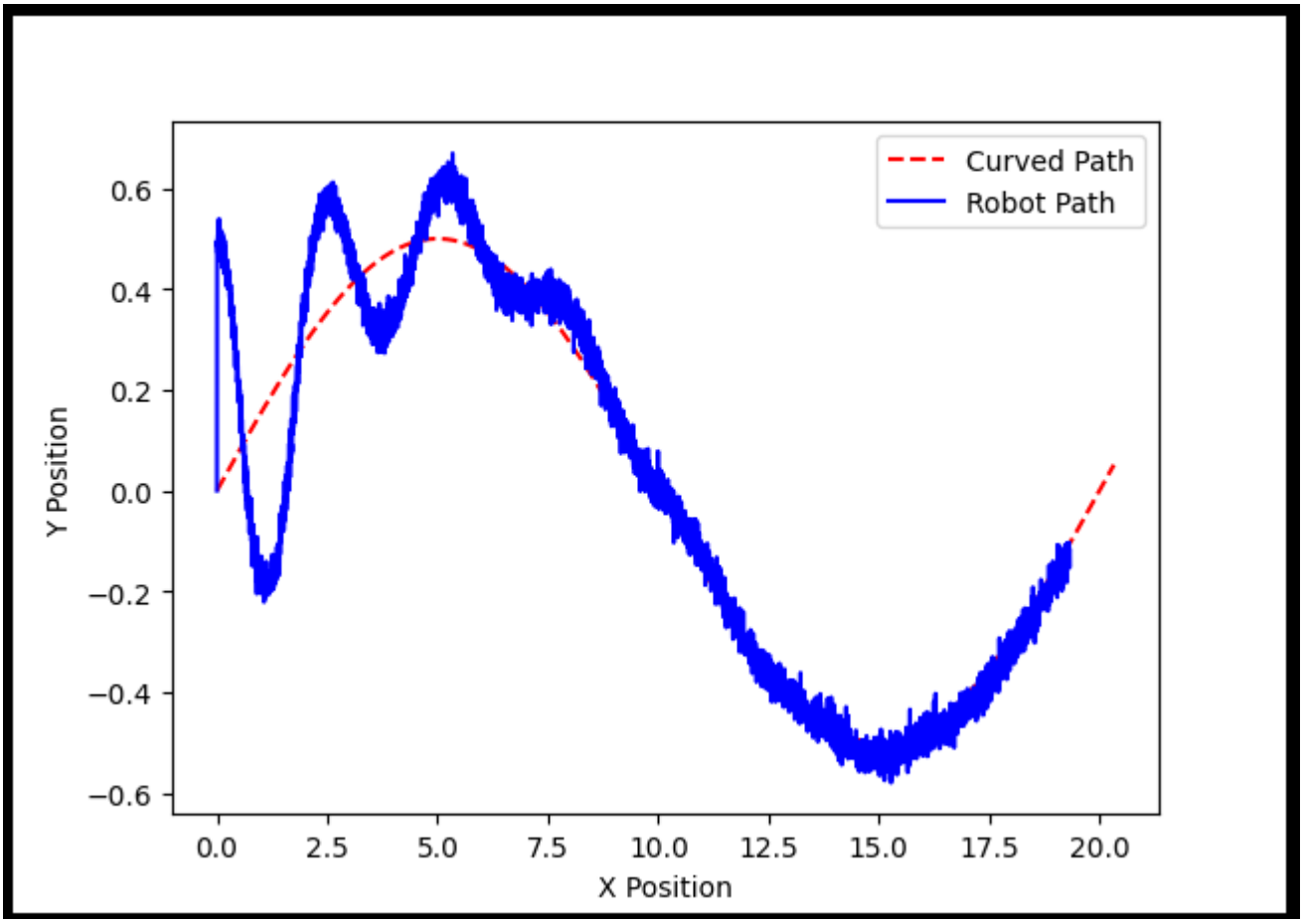
3. E3 - Noise and Disturbance Rejection:

- Kp =2 ki=0.001 kd=4
- noise levels=0.005



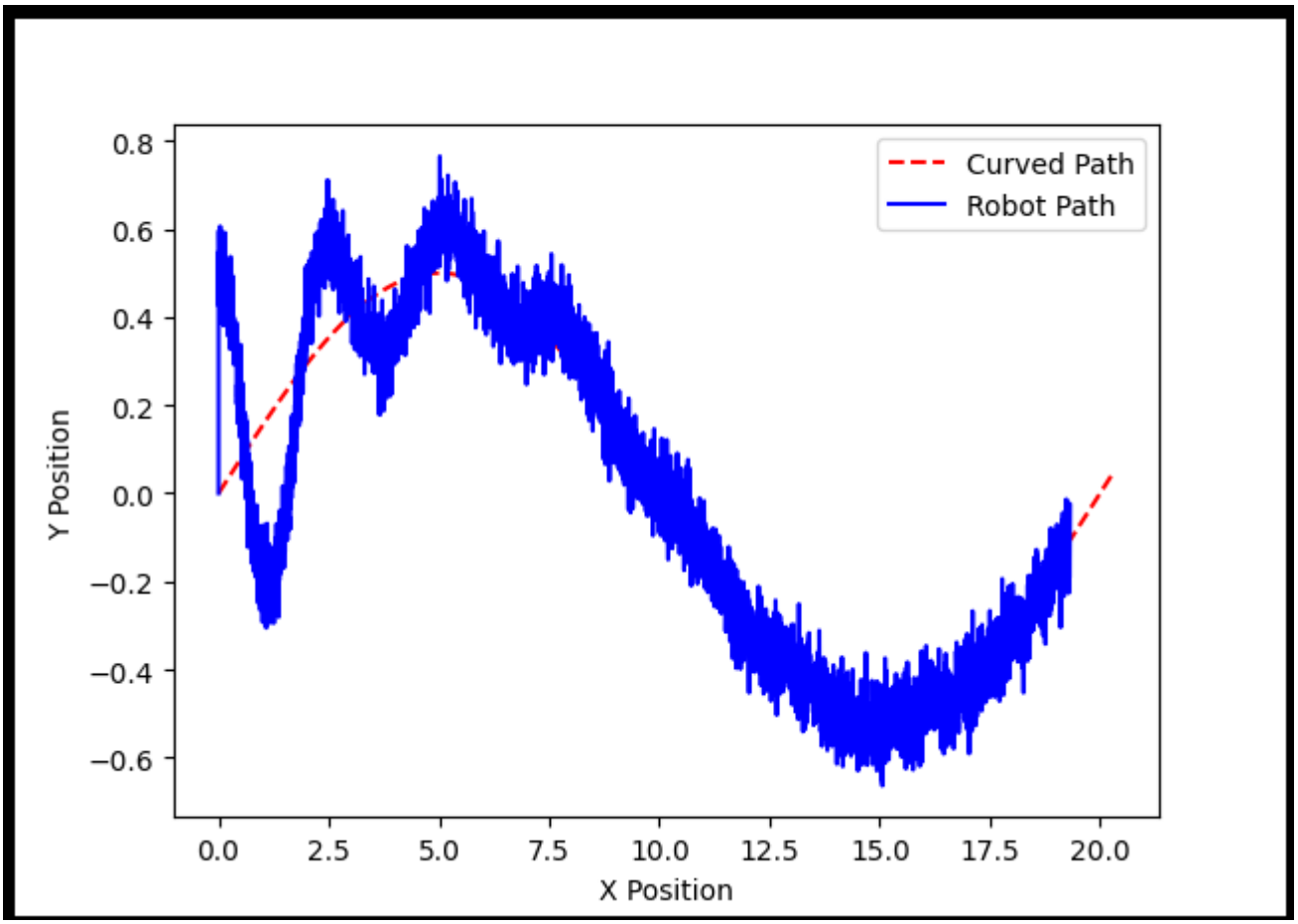

```
Overshoot: 0.5072 m
Steady State Error: 0.0099 m
Settling Time: 22.64 s
```

- noise levels=0.02




```
Overshoot: 0.5318 m
Steady State Error: 0.0137 m
Settling Time: 49.83 s
```

- noise levels=0.05




```
Overshoot: 0.6004 m
Steady State Error: 0.0791 m
Settling Time: Did not settle
```

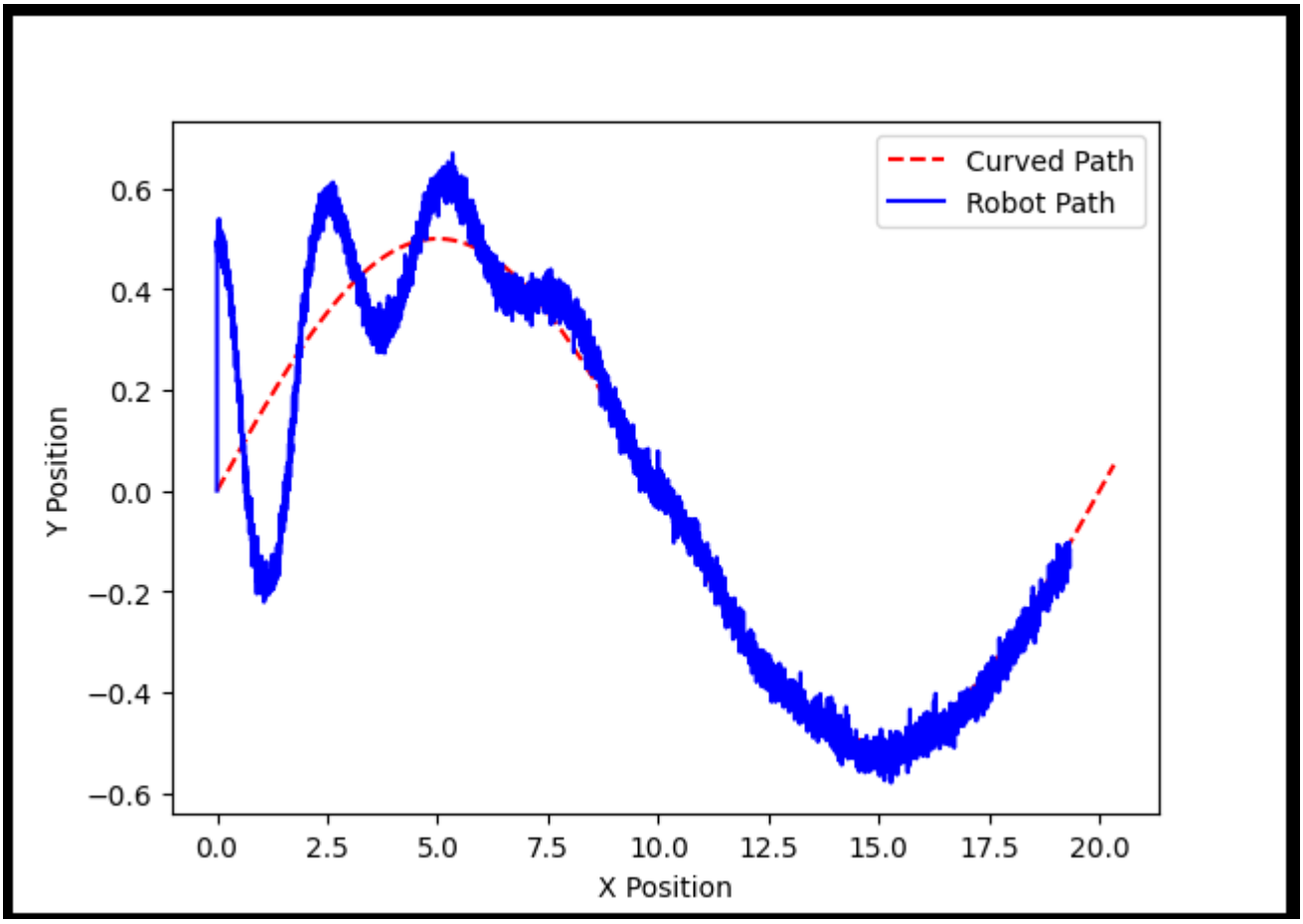
Noise	Overshoot	SS Error	Ts
0.005	0.5072 m	0.0099 m	22.64 s
0.02	0.5318 m	0.0137 m	49.83 s
0.05	0.6004 m	0.0791 m	Did not settle

Discussion:

As noise level increases, both overshoot and steady-state error increase. At noise=0.005, the robot successfully tracks the path with Ts=22.64s. At noise=0.02, performance degrades with Ts=49.83s. At noise=0.05, the robot fails to settle entirely, demonstrating that the PID controller has a noise tolerance limit beyond which it cannot maintain stable tracking.

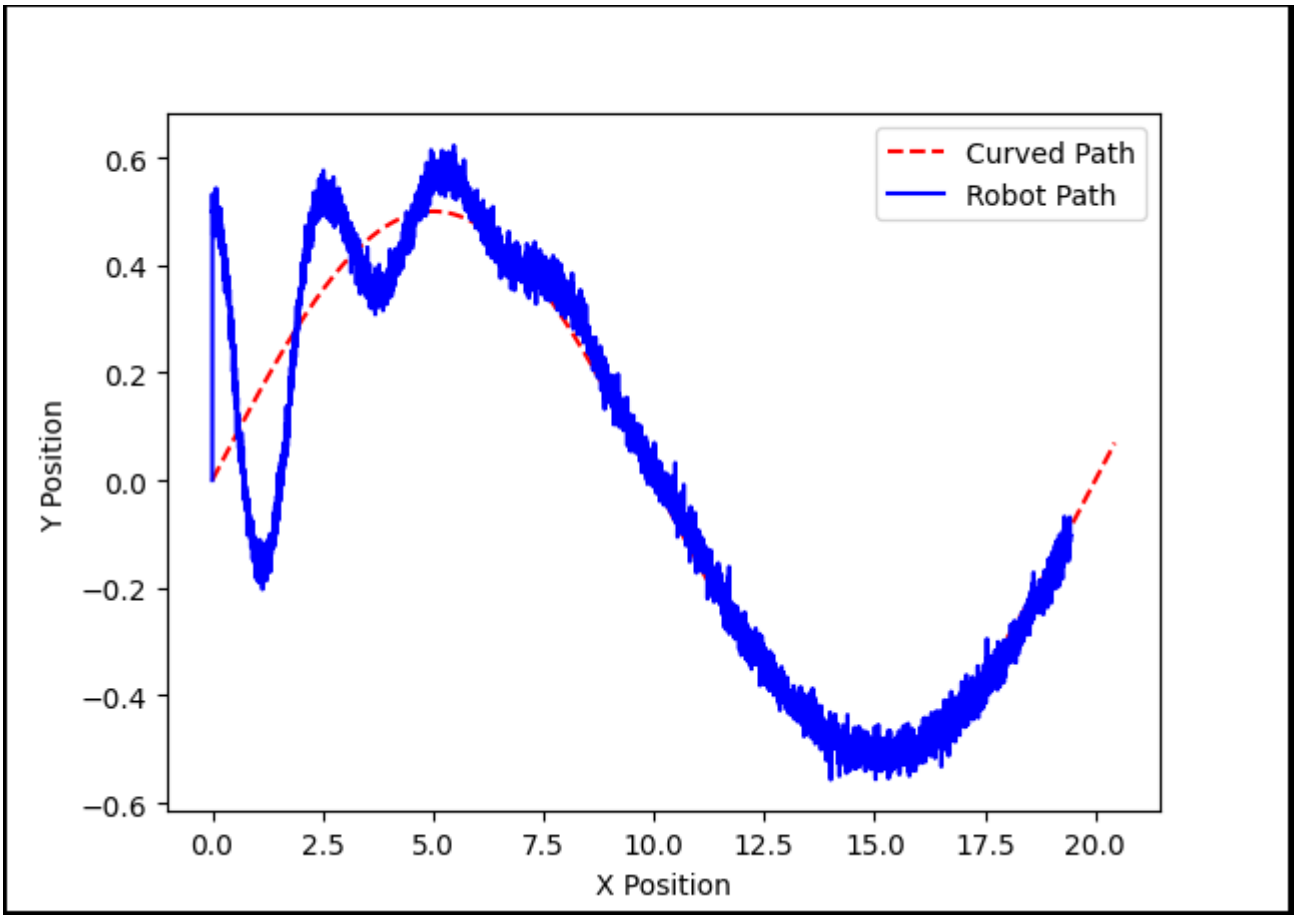
4. E4 - PD vs PID Ablation:

- noise levels=0.02
- Kp =2 ki=0.001 kd=4




```
Overshoot: 0.5318 m
Steady State Error: 0.0137 m
Settling Time: 49.83 s
```

- Kp =2 ki=0.00 kd=4




```
Overshoot: 0.5315 m
Steady State Error: 0.0152 m
Settling Time: 49.89 s
```

Controller	Overshoot	SS Error	Ts
PID (Ki=0.001)	0.5318 m	0.0137 m	49.83 s
PD (Ki=0)	0.5315 m	0.0152 m	49.89 s

Discussion:

The PD controller (Ki=0, SS Error=0.0152m) and PID controller (Ki=0.001, SS Error=0.0137m) showed similar performance under noise. The small Ki value provided marginal improvement in steady-state error. However, on the curved path under noise, the integral term did not significantly help because the reference signal is continuously changing, preventing integral accumulation from being beneficial.

7. Conclusion:

This project successfully implemented a line-following robot using PID control within a three-client VSI digital twin architecture. Results show that Kp=2.0, Ki=0.001, Kd=4.0 provides optimal performance on straight paths. Curved paths increase overshoot due to dynamic reference tracking. Noise levels above 0.02 significantly degrade controller performance. The PD and PID controllers show similar performance on curved paths under noise, suggesting that the derivative term is more critical than the integral term for this application.

Screencast: [Google Drive Link]

Source Code: https://github.com/mariamamr22004-dot/LineFollowerRobot